

Volet D : Ingénierie Logiciel et Data Engineering

OBJECTIF DU VOLET

Construire la plateforme qui permet :

- récupérer les données
- les traiter
- les stocker
- exposer les résultats
- faire tourner le modèle

1- ARCHITECTURE TECHNIQUE

Sources de données

(Compteurs + Météo)



Ingestion Batch

(Python)



Pipeline de traitement ETL

(Nettoyage + Features)



Stockage

(Stockage intermédiaire (Data Lake simplifié basé sur des fichiers CSV nettoyés))



Modèle IA

(Gradient Boosting)



API FastAPI



Docker



Dashboard Power BI

L'architecture repose sur une chaîne de traitement permettant de collecter les données de consommation et de météo, de les préparer pour l'apprentissage automatique puis de mettre les résultats à disposition des utilisateurs via une API et

un tableau de bord. Cette architecture est modulaire. Chaque composant peut évoluer indépendamment sans impacter les autres parties du système.

2- PIPELINE D'INGESTION (DATA ENGINEERING)

Le pipeline d'ingestion est réalisé en mode batch. Les données provenant des relevés de compteurs et des données météorologiques sont chargées depuis des fichiers CSV. Une étape de nettoyage est ensuite appliquée afin de supprimer les doublons et les valeurs incohérentes. Les données nettoyées sont enregistrées dans un Data Lake simplifié avant d'être exploitées par les traitements analytiques et les modèles de machine learning.

3- Exposition des données et services (Une ou plusieurs API)

Une API FastAPI a été développée afin d'exposer le modèle de prévision. Le modèle entraîné et sauvegardé au format Joblib est chargé automatiquement par l'API et peut être interrogé via un endpoint REST retournant une prévision de consommation énergétique.

4- Déploiement

Le déploiement du modèle a été réalisé via Docker afin de garantir la portabilité et la reproductibilité de l'application.

Une API REST a été développée avec FastAPI, permettant d'exposer le modèle de Machine Learning sous forme de service accessible via HTTP. Cette API charge le modèle préalablement entraîné et sauvegardé au format .pkl, puis retourne des prédictions à partir de données en entrée.

L'ensemble de l'application (code, dépendances, modèle) a été encapsulé dans un conteneur Docker à l'aide d'un fichier Dockerfile. Cela permet de déployer facilement la solution sur n'importe quel environnement sans dépendances externes.

Cette approche s'inscrit dans une logique MLOps en facilitant le passage du modèle de la phase de développement à la production.

Tester l'API : <http://localhost:8000/docs>

5- Intégration

L'ensemble des composants développés s'intègre dans une chaîne de traitement unique permettant d'automatiser le cycle de vie des données.

- Le pipeline d'ingestion récupère les données de consommation et les données météorologiques.
- Les données sont nettoyées et stockées dans le Data Lake sous forme de fichiers CSV préparés pour l'analyse.
- Le script d'entraînement charge les données du Data Lake et construit le modèle de prévision de consommation énergétique.
- Le modèle entraîné est sauvegardé dans le dossier models au format Joblib (best_model.pkl).
- L'API FastAPI charge automatiquement ce modèle afin de fournir des prédictions à la demande.
- L'application est conteneurisée avec Docker afin de faciliter son déploiement et sa reproductibilité.
- Les prédictions peuvent ensuite être exploitées par un tableau de bord Power BI ou toute autre application cliente.

donnees/

|

|— releves_consommation.csv

|— meteo.csv

|



ingestion.py



data_lake/

|

|— releves_clean.csv

|— meteo_clean.csv

|



train_model.py



models/

|

|— best_model.pkl

|



FastAPI

(app.py)



Docker



Power BI / Utilisateur